SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

Collaborative Study Material & Quiz Platform (CSMQP)

System Design Document
Version: 1.0
Governing Standard: IEEE Std 1016-2009

---

1. Architectural Design

1.1 Architectural Pattern

The Collaborative Study Material and Quiz Platform (CSMQP) utilizes a **Layered (3-Tier) Architecture** coupled with an **Event-Driven Subsystem** to achieve low-latency real-time quiz synchronization and clear separation of concerns:

- **Presentation Tier:** Built as a Single Page Application (SPA) using React and Tailwind CSS. Renderings for analytical data are managed via Recharts/Chart.js, while live interaction is handled through WebSocket client connections.
- **Application / Service Tier:** Consists of RESTful API endpoints for state management and an asynchronous WebSocket room manager (Node.js/FastAPI). It also houses an analytical aggregation engine for background metrics processing.
- **Data & Storage Tier:** Utilizes PostgreSQL 15 as the primary relational database with B-Tree indexing. Binary documents and uploaded study assets are stored using S3-compatible object storage.

1.2 Traceability to Requirements

Module / Component	Target Requirement	Architectural Implementation
Material Service	FR-02	Manages multipart file streams, enforces file size validation, and controls object storage reference pointers.

Module / Component	Target Requirement	Architectural Implementation
Question Bank Service	FR-03	Handles relational question pools, metadata/tag filtering, and server-side answer key validation logic.
WebSocket Quiz Engine	FR-04, NFR-02, SEC-02	Manages ephemeral room states, executes low-latency (<= 150 ms) message fan-out, and performs server-side masking and withholding of quiz answer keys until round completion.
Analytics Engine	FR-05, NFR-01	Runs background aggregation queries using optimized database indexes over attempt logs.
Security Middleware	SEC-01	Enforces stateless JWT authentication, RBAC authorization, and input validation.

1.3 Security Architecture

- **Perimeter Security:** Nginx operates as a reverse proxy and SSL/TLS termination gateway. It enforces strict rate limiting (maximum 120 requests/minute per IP) to prevent brute-force attacks and Denial-of-Service attempts.
- **Transport Security:** Transport Layer Security (TLS 1.3) is enforced across all external public communication channels.
- **Authentication & Authorization:** Uses stateless HS256 (HMAC-SHA256) signed JSON Web Tokens (JWTs) delivered via secure, HTTP-only cookies to prevent cross-site scripting (XSS) token extraction. Passwords are stored using Bcrypt password hashing (salted, work factor >= 12).
- **Data Sanitization:** Database operations utilize prepared statements with parameter binding to prevent SQL injection vulnerabilities.

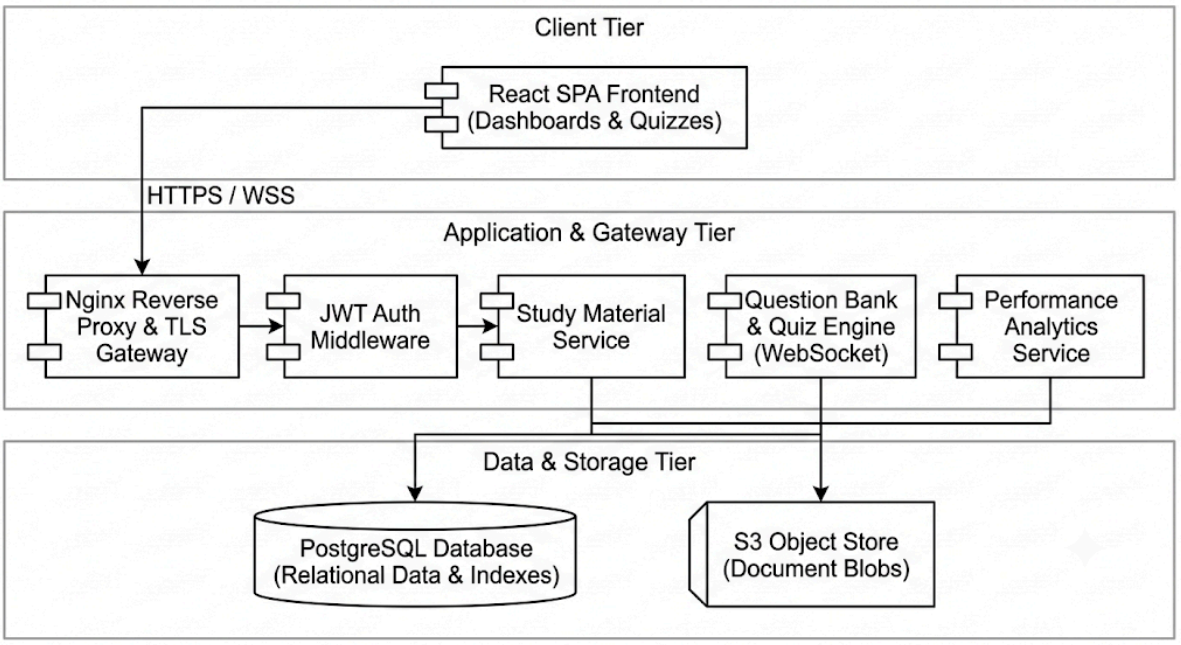
---

2. Component Design & Structural Overview

2.1 Component Breakdown

The platform consists of nine core system components working in tandem across the network boundary:

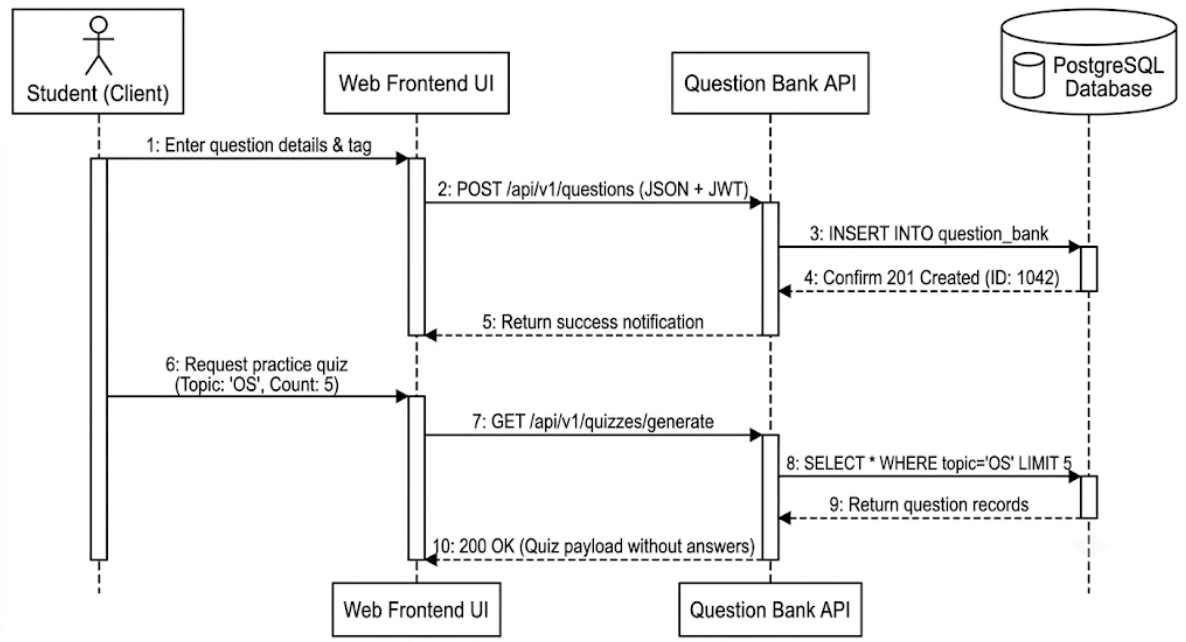
- **Web Frontend SPA:** Provides user dashboards, file uploads, quiz execution environments, and real-time scoreboards.
- **Nginx Reverse Proxy:** Manages TLS termination, load routing, rate limiting, and static resource distribution.
- **Auth Middleware:** Intercepts REST and WebSocket handshakes to validate JWT integrity and verify Role-Based Access Control (RBAC) permissions.
- **Material Service:** Handles file parsing, upload stream validation, object storage writes, and access link generation.
- **Question Bank Service:** Enforces schema validation for user-submitted questions and provides tagged search capabilities.
- **WebSocket Quiz Engine:** Coordinates active room lifecycles, runs server-side countdown timers, processes synchronized answer submissions, and broadcasts live leaderboards.
- **Performance Analytics Service:** Processes historical attempt logs to extract topic-wise accuracy trends and percentile distributions.
- **PostgreSQL Database:** Provides persistent relational storage for user profiles, courses, questions, attempts, and platform indexes.
- **Object Storage:** Offers secure blob storage for binary files (e.g., PDF notes, slide decks) with signed access URLs.



---

3. Sequence & Behavioral Design

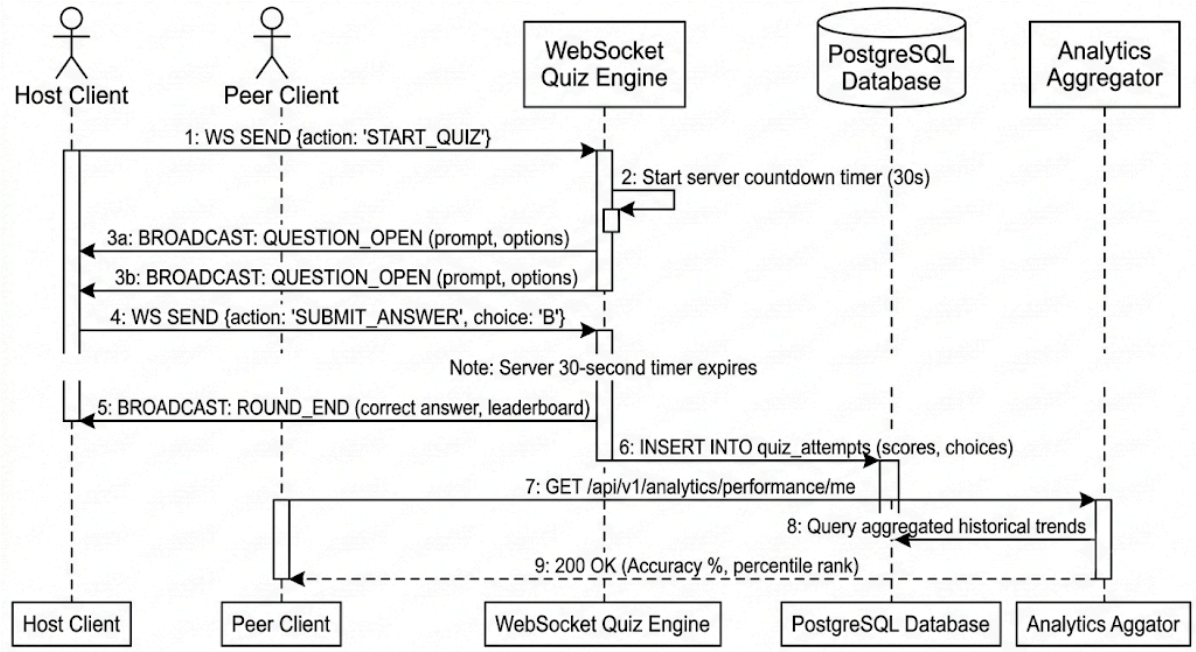
3.1 Sequence Flow 1: Question Bank Contribution & Practice Quiz Generation



1. **User Action:** An authorized user inputs a question prompt, multiple-choice options, correct choice index, topic tag, and explanation.
2. **Frontend Dispatch:** The client issues a `POST /api/v1/questions` request containing the question structure and bearer token.

3. **Validation & Storage:** The API server checks the authorization payload, validates input schemas, and writes the record to the `question_bank` table.
4. **Confirmation:** Upon successful insertion, the API server returns a `201 Created` status with the system-assigned question ID.
5. **Quiz Generation:** The user requests a practice session by specifying a topic and target question count via `GET /api/v1/quizzes/generate`.
6. **Query Execution:** The database selects matching records using an indexed random selection query and returns the structured question set without revealing answer keys.

3.2 Sequence Flow 2: Synchronized Quiz Room Execution & Analytics Ingestion



1. **Room Initialization:** Participants establish connection to a common room endpoint over WebSockets.
2. **Start Event:** The room host sends a `START_QUIZ` payload. The WebSocket engine initializes a server-side 30-second timer and broadcasts `QUESTION_OPEN` data to all connected clients.
3. **Participant Answer:** Clients emit `SUBMIT_ANSWER` with the user choice before the timer reaches zero.
4. **Round Completion:** Upon timer expiration, the server calculates round scores and broadcasts `ROUND_END` with the correct answer and updated standings.
5. **Persistence:** The engine writes attempt data and aggregated user scores to the `quiz_attempts` table.

6. **Analytics Ingestion:** When navigating to the metrics tab, the frontend requests `GET /api/v1/analytics/performance/me` to compute accuracy breakdowns across historical attempts.

---

4. API Design & Error Handling

4.1 Core Endpoint Specifications

POST `/api/v1/questions`

Creates a new question entry in the global question bank.

- **Headers:**
  - `Authorization`: `Bearer <JWT_TOKEN>`
  - `Content-Type`: `application/json`
- **Request Body:**

```
{
  "topic": "Computer Networks",
  "prompt": "What is the primary function of ARP?",
  "options": [
    "Map IP to MAC",
    "Resolve DNS",
    "TCP Handshake",
    "Routing"
  ],
  "correct_index": 0,
  "explanation": "ARP maps IP addresses to physical MAC addresses."
}
```

- **Response (`201 Created`):**


```
{
  "question_id": 1042,
  "status": "published"
}
```


GET `/api/v1/analytics/performance/me`

Retrieves aggregated performance analytics for the authenticated user.

- **Headers:**
  - `Authorization`: `Bearer <JWT_TOKEN>`
- **Response (`200 OK`):**


```
{
  "user_id": 88,
  "total_quizzes_taken": 22,
  "cumulative_accuracy_percentage": 81.5,
  "cohort_percentile": 89.4,
  "topic_breakdown": [
    {
      "topic": "Computer Networks",
      "accuracy": 90.0
    }
  ]
}
```

GET `/api/v1/analytics/performance/{user_id}`

Retrieves analytics for a target user ID. Restricted to the user themselves or users with Admin/Instructor roles; requests by other students return 403 Forbidden.

- **Headers:**
  - `Authorization`: `Bearer <JWT_TOKEN>`
- **Response (`200 OK` or `403 Forbidden`):**


```
{
  "user_id": 88,
  "total_quizzes_taken": 22,
  "cumulative_accuracy_percentage": 81.5,
  "cohort_percentile": 89.4,
  "topic_breakdown": [
    {
      "topic": "Computer Networks",
      "accuracy": 90.0
    }
  ]
}
```


4.2 Error Handling Strategy

The application uses standardized HTTP response structures adhering to **RFC 7807 Problem Details** specifications. Unhandled system exceptions are intercepted by global middleware filters to produce consistent client outputs without exposing database or server stack traces.

Standard Error Response Format


```
{
```

```
  "type": "https://csmqp.net/errors/validation-error",
  "title": "Invalid Request Payload",
  "status": 400,
  "detail": "The field 'correct_index' must be an integer between 0 and 3.",
  "instance": "/api/v1/questions",
  "timestamp": "2026-09-29T10:15:30Z"
}
```